ECEN-4243 Lab 2: Single-Cycle RISC-V Microarchitecture

Landon Schultz (CWID: A20416726)
Kiaan Doolabh (CWID: A20429908)

April 3, 2026

1 Introduction

The main goal of this lab was to build a working RISC-V processor in system verilog, and then implement it onto an FPGA. We started the lab with a partial design of the processor and had to finish all the rest of the functions that you would commonly have on a RISC-V processor. This single cycle microarchitecture we created was made to correctly fetch, decode, and execute commands, as well as accessing and writing the results back to memory. We implemented and verified the instructions one by one and then also worked to verify that the code actually worked when incorporated into an FPGA. Most of the lab was debugging, as we had to trace signals through the datapath of a processor that we have talked about in class many times up to this point.

2 Design

2.1 System Overview

The processor we implemented is based on a single cycle RV32I structure as stated above. This means that each instruction was to be completed in a single cycle of the clock, so we had to fetch the instruction, decode it, execute it, access the memory addresses of things in the instruction, and then write back to the register the outcome of the instruction all in a single pass through our datapath. Here are the main blocks that we were working with, along with an image from the slides showing how the datapath is all linked together. Our design followed the Harvard style memory setup in the simulation, meaning we had separate instruction and data memory, making it much easier to debug the load and store type operations.

- Program Counter (PC)
- Instruction Memory
- Register File
- ALU
- Immediate Extension Logic
- Data Memory
- Main Decoder / Control Logic
- Branch and Jump Control Logic

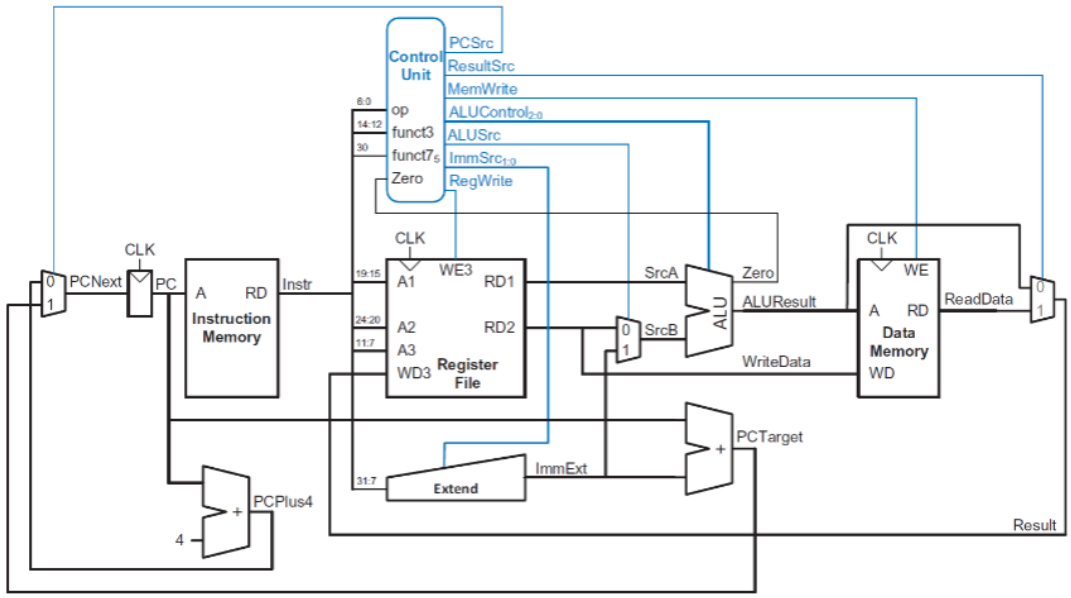

Figure 1: Single-cycle RISC-V datapath structure used as the basis for the lab implementation.

2.2 Instruction Support

For this lab, we had to expand the starter processor so that it could support a much larger group of RV32I instructions. We mainly verified this by building our own large test file that exercised a large amount of the commands in one run, while also checking some commands individually along the way.

The instruction groups that we added and tested included:

- **Arithmetic:** `add, sub`
- **Upper / PC-relative:** `lui, auipc`
- **Logical:** `and, or, xor`
- **Shift:** `sll, srl, sra, slli, srli, srai`
- **Comparison:** `slt, sltu`
- **Immediate:** `andi, ori, xori`
- **Load / Store:** `lw, sw, lb, lbu, lh, sb, sh`
- **Branch:** `beq, bne, blt, bge, bltu, bgeu`
- **Jump:** `jal, jalr`

This meant that we had to expand both the datapath and the controller a lot from the starter design we were given.

2.3 Datapath Changes

One of the main parts of the lab was correctly implementing the whole datapath so that the correct values were called upon and worked with based on the instruction that the user wanted to use. Some great examples of big updates we had to make was supporting immediate in the Arithmetic Logic Unit (ALU) instructions, allowing PC relative instructions like auipc and jal, supporting jump target generation for jalr, getting immediate support for all the immediate type instructions, and adding correct sub word memory instruction like byte and half word loads and stores.

2.4 Control Logic Changes

In order for the updated datapath to work correctly, we had to update the control logic so that the proper signals would be generated for all of the new instructions. This meant expanding the decoder so that it could look at the opcode, funct3, and funct7 bits to find and select the correct control outputs. Examples of this would be expanding the ALU from 3 to 4 bits so that it could support shifts and comparisons, controlling where the write was going to, handling branching decisions, and so forth.

2.5 Memory Support Changes

This is one of the bigger changes that was quite hard to implement. For this lab, we needed to add the following sub word and word memory acess:

- Byte stores (`sb`)
- Halfword stores (`sh`)
- Word stores (`sw`)
- Byte loads (`lb, lbu`)
- Halfword loads (`lh, lhu`)
- Word loads (`lw`)

To get these to work, we had to add in some logic in the data memory module to write only the correct byte or halfword depending on the address' offset. We also added a loadext module so that the load instruction would have a correct extened value depending on if it was signed or not and based on the load type.

3 Implementation

3.1 System Verilog Work

The implementation for this lab was done directly inside the provided riscv$_{s}ingle.dofile, wherewemodifiedthefollo$

- `maindec`
- `controller`
- `aludec`

- datapath
- branchcomp
- loadext
- extend
- dmem
- imem

3.2 Example Code Changes

Here are some examples of code that we changed so that you can actually see where we made updates to our code. This first example is where we extended the immediate extension unit so that we could support all of the instruction formats that we would need. If we did not make this update, most of the immediate functions would fail and throw errors. Here is the updated code to show how each format is identified and read.

Listing 1: Immediate extension logic used to support multiple RV32I instruction formats.

```
always_comb
  case(immsrc)
    3'b000:  immext = {{20{instr[31]}}, instr[31:20]};
    3'b001:  immext = {{20{instr[31]}}, instr[31:25], instr[11:7]};
    3'b010:  immext = {{20{instr[31]}}, instr[7], instr[30:25], instr
        [11:8], 1'b0};
    3'b011:  immext = {{12{instr[31]}}, instr[19:12], instr[20], instr
        [30:21], 1'b0};
    3'b100:  immext = {instr[31:12], 12'b0};
    default: immext = 32'bx;
  endcase
```

Another very important update was the branch comparison logic. Instead of trying to get everything to run through the ALU, we instead use a seperate branch comparison module to determine whether a branch should be taken.

Listing 2: Branch comparison logic used for RV32I branch instructions.

```
always_comb
  case(funct3)
    3'b000:  result = (a == b);
    3'b001:  result = (a != b);
    3'b100:  result = ($signed(a) < $signed(b));
    3'b101:  result = ($signed(a) >= $signed(b));
    3'b110:  result = (a < b);
    3'b111:  result = (a >= b);
    default: result = 1'b0;
  endcase
```

We also expanded the ALU so that it could support more operations than just the original basic arithmetic. This included logic operations, shifts, signed and unsigned comparisons, and subtraction.

```
VSIM 4> examine -hex /testbench/dut/rv32single/dp/rf/rf[31]
# 32'hxxxxxxxx
VSIM 5> examine -hex /testbench/dut/rv32single/dp/rf/rf[30]
# 32'hxxxxxxxx
VSIM 6> examine -hex /testbench/dut/rv32single/dp/rf/rf[29]
# 32'hxxxxxxxx
VSIM 7> examine -hex /testbench/dut/rv32single/dp/rf/rf[28]
# 32'hxxxxxxxx
VSIM 8> examine -hex /testbench/dut/rv32single/dp/rf/rf[27]
# 32'hxxxxxxxx
VSIM 9> examine -hex /testbench/dut/rv32single/dp/rf/rf[26]
# 32'hxxxxxxxx
VSIM 10> examine -hex /testbench/dut/rv32single/dp/rf/rf[25]
# 32'hxxxxxxxx
VSIM 11> examine -hex /testbench/dut/rv32single/dp/rf/rf[24]
# 32'hxxxxxxxx
VSIM 12> examine -hex /testbench/dut/rv32single/dp/rf/rf[23]
# 32'hxxxxxxxx
VSIM 13> examine -hex /testbench/dut/rv32single/dp/rf/rf[22]
# 32'hxxxxxxxx
VSIM 14> examine -hex /testbench/dut/rv32single/dp/rf/rf[21]
# 32'h00001000
VSIM 15> examine -hex /testbench/dut/rv32single/dp/rf/rf[20]
# 32'h091a2b3c
VSIM 16> examine -hex /testbench/dut/rv32single/dp/rf/rf[19]
# 32'h091a2b3c
VSIM 17> examine -hex /testbench/dut/rv32single/dp/rf/rf[18]
# 32'h48d159e0
VSIM 18> examine -hex /testbench/dut/rv32single/dp/rf/rf[17]
# 32'h123456d2
VSIM 19> examine -hex /testbench/dut/rv32single/dp/rf/rf[16]
# 32'h123456f8
VSIM 20> examine -hex /testbench/dut/rv32single/dp/rf/rf[15]
# 32'h00000008
VSIM 21> examine -hex /testbench/dut/rv32single/dp/rf/rf[14]
# 32'h00000000
VSIM 22> examine -hex /testbench/dut/rv32single/dp/rf/rf[13]
# 32'h00000000
VSIM 23> examine -hex /testbench/dut/rv32single/dp/rf/rf[12]
# 32'hf9abcdef
VSIM 24> examine -hex /testbench/dut/rv32single/dp/rf/rf[11]
# 32'h09abcdef
VSIM 25> examine -hex /testbench/dut/rv32single/dp/rf/rf[10]
# 32'h91a2b3c0
VSIM 26> examine -hex /testbench/dut/rv32single/dp/rf/rf[9]
# 32'h88888888
```

Figure 2: Shows testing and results of each instruction

4 Testing Strategy

Now to the fun part, making sure all the updates we made actually work. We did our testing through MODELSIM, and since we added a large amount of instructions, it was not enough to just run one test code and make sure that it worked. We took the strategy of testing as we went along, making sure that the instructions worked as we implemented them. After this, we used an all encompassing test that was included to make sure all the functions worked in tandem.

4.1 Individual Verification Test Examples

This is a log of verification tests done in MODELSIM for the instructions:

4.2 Main Verification Test

Our main confirmation test was the `test_hw.S` file we were given. This seemed designed to hit all of the instructions that we implemented, while also loading recognizable values into the registers so we could go through and make sure the registers were behaving correctly. The test first loaded in x1 and x2 into memory, then used those values to test arithmetic, logical, shift, comparison, immediate, memory, branch, and then jump instructions. This made it quite easy to see all at once if our updated we made were truly all working.

Listing 3: Excerpt from the custom RV32I verification test used in simulation.

```
li x1, 0x12345678
li x2, 0x9ABCDEF0

add x3, x1, x2
sub x4, x2, x1
```

```
lui x5, 0x10000
auipc x6, 0

and x7, x1, x2
or  x8, x1, x2
xor x9, x1, x2

slli x18, x1, 2
srli x19, x1, 1
srai x20, x1, 1

sw x1, 0(x21)
lw x22, 0(x21)
```

5 Simulation Results

5.1 Successful Simulation Run

When running our simulation in modelsim, one of the first things we looked for was that the simulation actually ran and put out a good looking waveform As seen below, this modelsim transcript shows one of our successful modelsim runs of the large testhw.S file. This shows that the program was able to go through the provided test file and hit the expected outcome on the testbench.

```
#
# Top level modules:
#       testbench
#       flopenr
# End time: 15:18:15 on Mar 23,2026, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0
# End time: 15:18:19 on Mar 23,2026, Elapsed time: 0:00:39
# Errors: 1, Warnings: 0
# vsim -debugdb -voptargs="+acc" work.testbench
# Start time: 15:18:19 on Mar 23,2026
# ** Note: (vsim-8009) Loading existing optimized design _opt
# Loading sv_std.std
# Loading work.testbench(fast)
# Loading work.top(fast)
# Loading work.riscvsingle(fast)
# Loading work.controller(fast)
# Loading work.maindec(fast)
# Loading work.aludec(fast)
# Loading work.datapath(fast)
# Loading work.flopr(fast)
# Loading work.adder(fast)
# Loading work.mux2(fast)
# Loading work.regfile(fast)
# Loading work.extend(fast)
# Loading work.mux3(fast)
# Loading work.alu(fast)
# Loading work.branchcomp(fast)
# Loading work.loadext(fast)
# Loading work.imem(fast)
# Loading work.dmem(fast)
# ** Note: (vsim-8900) Creating design debug database vsim.dbg.
# ** Error: (vsim-8357) Internal debugDB error id:ctl  ret:1 table context_tbl already exists
# Simulation succeeded
# ** Note: $stop    : riscv_single.sv(86)
#    Time: 195 ns  Iteration: 1  Instance: /testbench
```

Figure 3: ModelSim transcript showing a successful processor simulation run.

5.2 Register Verification in ModelSim

After this run of the simulation, we instepected the register file directly using modelsim. Doing this, we were bale to verify the process by directly comparing the output register values to the known values for these registers. The following image shows these results, where we used the examine -hex comand to inspect the registers manually after the sim finished.

```
VSIM 4> examine -hex /testbench/dut/rv32single/dp/rf/rf[31]
# 32'hxxxxxxxx
VSIM 5> examine -hex /testbench/dut/rv32single/dp/rf/rf[30]
# 32'hxxxxxxxx
VSIM 6> examine -hex /testbench/dut/rv32single/dp/rf/rf[29]
# 32'hxxxxxxxx
VSIM 7> examine -hex /testbench/dut/rv32single/dp/rf/rf[28]
# 32'hxxxxxxxx
VSIM 8> examine -hex /testbench/dut/rv32single/dp/rf/rf[27]
# 32'hxxxxxxxx
VSIM 9> examine -hex /testbench/dut/rv32single/dp/rf/rf[26]
# 32'hxxxxxxxx
VSIM 10> examine -hex /testbench/dut/rv32single/dp/rf/rf[25]
# 32'hxxxxxxxx
VSIM 11> examine -hex /testbench/dut/rv32single/dp/rf/rf[24]
# 32'hxxxxxxxx
VSIM 12> examine -hex /testbench/dut/rv32single/dp/rf/rf[23]
# 32'hxxxxxxxx
VSIM 13> examine -hex /testbench/dut/rv32single/dp/rf/rf[22]
# 32'hxxxxxxxx
VSIM 14> examine -hex /testbench/dut/rv32single/dp/rf/rf[21]
# 32'h00001000
VSIM 15> examine -hex /testbench/dut/rv32single/dp/rf/rf[20]
# 32'h091a2b3c
VSIM 16> examine -hex /testbench/dut/rv32single/dp/rf/rf[19]
# 32'h091a2b3c
VSIM 17> examine -hex /testbench/dut/rv32single/dp/rf/rf[18]
# 32'h48d159e0
VSIM 18> examine -hex /testbench/dut/rv32single/dp/rf/rf[17]
# 32'h123456d2
VSIM 19> examine -hex /testbench/dut/rv32single/dp/rf/rf[16]
# 32'h123456f8
VSIM 20> examine -hex /testbench/dut/rv32single/dp/rf/rf[15]
# 32'h00000008
VSIM 21> examine -hex /testbench/dut/rv32single/dp/rf/rf[14]
# 32'h00000000
VSIM 22> examine -hex /testbench/dut/rv32single/dp/rf/rf[13]
# 32'h00000000
VSIM 23> examine -hex /testbench/dut/rv32single/dp/rf/rf[12]
# 32'hf9abcdef
VSIM 24> examine -hex /testbench/dut/rv32single/dp/rf/rf[11]
# 32'h09abcdef
VSIM 25> examine -hex /testbench/dut/rv32single/dp/rf/rf[10]
# 32'h91a2b3c0
VSIM 26> examine -hex /testbench/dut/rv32single/dp/rf/rf[9]
# 32'h88888888
```

Figure 4: Register values examined directly in ModelSim after execution of the verification test.

5.3 Verification Table

Finally, we made an easy to read verification table that allows you to better see how each of the instructions in the testbench fared. This figure makes it much easier to see how many of the implemented instructions were producing the correct results.

test_hw.memfile — Register Verification Results

Register	Value	Expected	Instruction	Status
x0	0x00000000	0x00000000	hardwired zero	PASS
x1	0x12345678	0x12345678	lui + addi	PASS
x2	0x9ABCDEF0	0x9ABCDEF0	lui + addi	PASS
x3	0xACF13568	0xACF13568	add x3, x1, x2	PASS
x4	0x88888878	0x88888878	sub x4, x2, x1	PASS
x5	0x10000000	0x10000000	lui x5, 0x10000	PASS
x6	0x0000001C	0x0000001C	auipc x6, 0	PASS
x7	0x12345670	0x12345670	and x7, x1, x2	PASS
x8	0x9ABCDEF8	0x9ABCDEF8	or x8, x1, x2	PASS
x9	0x88888888	0x88888888	xor x9, x1, x2	PASS
x10	0x91A2B3C0	0x91A2B3C0	slli x10, x1, 3	PASS
x11	0x09ABCDEF	0x09ABCDEF	srli x11, x2, 4	PASS
x12	0xF9ABCDEF	0xF9ABCDEF	srai x12, x2, 4	PASS
x13	0x00000000	0x00000000	slt x13, x1, x2	PASS
x14	0x00000000	0x00000000	sltu x14, x2, x1	PASS
x15	0x00000008	0x00000008	andi x15, x1, 0xF	PASS
x16	0x123456F8	0x123456F8	ori x16, x1, 0xF0	PASS
x17	0x123456D2	0x123456D2	xori x17, x1, 0xAA	PASS
x18	0x48D159E0	0x48D159E0	slli x18, x1, 2	PASS
x19	0x091A2B3C	0x091A2B3C	srli x19, x1, 1	PASS
x20	0x091A2B3C	0x091A2B3C	srai x20, x1, 1	PASS
x21	0x00001000	0x00001000	auipc + addi (la)	PASS
x22	0x12345678	0x12345678	lw x22, 0(x21)	PASS

Figure: Register dump from test_hw.memfile simulation in ModelSim. Values obtained via examine -hex /testbench/dut/rv32single/dp/rf/rf[N]. All 23 observable registers match expected values, verifying correct operation of all RV32I instruction types.

Figure 5: Verification summary comparing expected and observed register values for the main test program.

5.4 Simulation Discussion

Overall, the simulation results gave us much stronger confidence that the processor was working correctly than just watching a waveform alone would have. By checking the final register contents

directly, we were able to verify that many of the implemented instructions were producing the correct results.

This was especially helpful for instructions like shifts, memory operations, and branch-related instructions, where it is easy for the code to compile and still be doing the wrong thing internally.

6 FPGA Implementation

6.1 Vivado Flow

After we verified everything in modelsim, we moved onto implementing the design on the FPGA. We had to make some changes to our riscvsingle.do file, with some of the major changes denoted below:

- Adding handshake-style signals such as `MemStrobe` and `PCReady` to better control memory access timing
- Modifying the program counter register to use an enable signal, preventing it from advancing when the system is not ready
- Updating the controller to generate memory strobe signals for load and store operations
- Expanding instruction memory size to support larger test programs
- Removing the simulation testbench and top module so the design could integrate with the Vivado project structure

These changes were important because FPGA execution requires more careful synchronization with memory and external interfaces than simulation does.

6.2 Hardware Challenges

Even after making these changes, we had some troubles with the hardware implimentation. Our code would successfully make it through the synthesis, implimentation, and generation of the bitstream, as well as go onto the FPGA with no issues. However, when we would run our processor on our FPGA, the PC would incriment up by 4 like it was supposed to, but all the other fields were returning straight zeros. Because of this, most of our confidence in our correctness in this lab comes from the modelsim output and testing, and not from the FPGA implementation. The FPGA implementation was producing an output after generating bitstream, and implementation was executing correctly, but output values were seemingly not correct. We tried many different ways to come up with a solution, but ended up falling short in this aspect. Like previously stated, we are confident that our code was operating correctly outside of a minor issue that likely occurred in our inclusion of files in vivado.

7 Analysis

The hardest part of this lab was definitely not just writing the code, but proving that the code was actually correct. Since the processor has so many pieces all interacting with each other, one small issue in the datapath or control logic can break the entire machine.

For us, the biggest struggle ended up being the FPGA implementation side of the lab rather than the instruction implementation itself. Once we had a stronger testing setup in ModelSim and

began checking register values directly, it became much easier to verify whether the instructions were behaving correctly. The simulation environment gave us much more confidence in the actual processor functionality.

The FPGA side, however, was much more difficult to debug. Even when the processor seemed like it was partially doing something, it was hard to tell whether the issue was in the processor itself, the memory initialization, the FPGA setup, or just the way the signals were being observed. From talking with others and seeing how much time was spent on this part, it seemed like this was a common difficulty for a lot of people in the lab.

This lab forced us to think about how instructions actually move through a processor instead of just seeing them as assembly commands on a screen. Things like branches, loads, stores, and jumps seem simple at the assembly level, but actually implementing them in hardware is much more involved.

One of the biggest takeaways from this lab was how important debugging and verification are in digital design. Just because code compiles does not mean it works. The processor only became believable once we could directly inspect the register values and verify that they matched what the instructions should have produced.

Probably the biggest thing this lab showed was that simulation and hardware implementation are not the same thing. A design can be much easier to prove in simulation than it is to fully validate on an FPGA, especially once memory setup and hardware debugging tools get involved.

8 Conclusion

Overall, this lab was a much more hands-on way of understanding how a processor actually works. Instead of just using RISC-V assembly, we were implementing the hardware that makes the instructions possible in the first place. We had to build out the datapath, expand the control logic, support more instruction types, and then verify that everything was working through simulation.

The biggest part of the lab ended up being debugging and verification, but that was also what made the lab useful. It gave a much better understanding of how the ISA connects to the actual hardware underneath it.

In the end, this lab gave a much better picture of how all of the main parts of a single cycle processor work together, and it also showed how important careful testing is when building digital systems.

9 References

References

[1] ECEN4243_Lab2.pdf

[2] ECEN4243_Lab2_FPGA_Implementation_Guide.pdf

[3] Sarah Harris and David Harris, *Digital Design and Computer Architecture, RISC-V Edition*, Elsevier, 2021.

[4] Note that generative AI such as ChatGPT and Claude AI were used in the formatting and presentation of this latex document